Data Loading, Storage and File Formats

Problem Statement: Analyzing Sales Data from Multiple File Formats Dataset: Sales data in multiple file formats (e.g., CSV, Excel, JSON) Description: The goal is to load and analyze sales data from different file formats, including CSV, Excel, and JSON, and perform data cleaning, transformation, and analysis on the dataset.

Tasks to Perform: Obtain sales data files in various formats, such as CSV, Excel, and JSON.

1. Load the sales data from each file format into the appropriate data structures or dataframes.
2. Explore the structure and content of the loaded data, identifying any inconsistencies, missing values, or data quality issues.
3. Perform data cleaning operations, such as handling missing values, removing duplicates, or correcting inconsistencies.
4. Convert the data into a unified format, such as a common dataframe or data structure, to enable seamless analysis.
5. Perform data transformation tasks, such as merging multiple datasets, splitting columns, or deriving new variables.
6. Analyze the sales data by performing descriptive statistics, aggregating data by specific variables, or calculating metrics such as total sales, average order value, or product category distribution.
7. Create visualizations, such as bar plots, pie charts, or box plots, to represent the sales data and gain insights into sales trends, customer behavior, or product performance.

Code :

```
import pandas as pd

csv_data = pd.read_csv('sales_data.csv')
```

Converting the csv to json & xlsx

```
# Save as JSON file
csv_data.to_json('sales_data.json', orient='records', lines=True)

# Save as Excel file
csv_data.to_excel('sales_data.xlsx', index=False)
```

Explore the structure and content of the loaded data, identifying any inconsistencies

Performing Task on csv dataset

csv_data.head()

	Date	Day	Month	Year	Customer_Age	Age_Group	Customer_Gender	Country	State	Product_Category	Sub_Catego
0	2013-11-26	26	November	2013	19	Youth (<25)	M	Canada	British Columbia	Accessories	Bike Rac
1	2015-11-26	26	November	2015	19	Youth (<25)	M	Canada	British Columbia	Accessories	Bike Rac
2	2014-03-23	23	March	2014	49	Adults (35-64)	M	Australia	New South Wales	Accessories	Bike Rac
3	2016-03-23	23	March	2016	49	Adults (35-64)	M	Australia	New South Wales	Accessories	Bike Rac
4	2014-05-15	15	May	2014	47	Adults (35-64)	F	Australia	New South Wales	Accessories	Bike Rac

csv_data.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 113036 entries, 0 to 113035
Data columns (total 18 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Date              113036 non-null  object
 1   Day               113036 non-null  int64
 2   Month             113036 non-null  object
 3   Year              113036 non-null  int64
 4   Customer_Age      113036 non-null  int64
 5   Age_Group         113036 non-null  object
 6   Customer_Gender   113036 non-null  object
 7   Country           113036 non-null  object
 8   State             113036 non-null  object
 9   Product_Category  113036 non-null  object
 10  Sub_Category      113036 non-null  object
 11  Product           113036 non-null  object
 12  Order_Quantity    113036 non-null  int64
 13  Unit_Cost         113036 non-null  int64
 14  Unit_Price        113036 non-null  int64
 15  Profit            113036 non-null  int64
 16  Cost              113036 non-null  int64
 17  Revenue           113036 non-null  int64
dtypes: int64(9), object(9)
memory usage: 15.5+ MB
```

csv_data.columns

```
Index(['Date', 'Day', 'Month', 'Year', 'Customer_Age', 'Age_Group',
       'Customer_Gender', 'Country', 'State', 'Product_Category',
       'Sub_Category', 'Product', 'Order_Quantity', 'Unit_Cost', 'Unit_Price',
       'Profit', 'Cost', 'Revenue'],
      dtype='object')
```

# Checking for null values
csv_data.isnull().sum()

```
Date                0
Day                 0
Month               0
Year                0
Customer_Age        0
Age_Group           0
Customer_Gender     0
Country             0
State               0
Product_Category    0
Sub_Category        0
Product             0
Order_Quantity      0
Unit_Cost           0
Unit_Price          0
Profit              0
Cost                0
Revenue             0
dtype: int64
```

# Checking for duplicates
csv_data.duplicated()

```
0         False
1         False
2         False
3         False
4         False
          ...
113031    False
113032    False
113033    False
113034    False
113035    False
Length: 113036, dtype: bool
```

Performing task on json dataset

# Load JSON file
json_data = pd.read_json('sales_data.json', lines=True)

Perform data cleaning operations, such as handling missing values, removing duplicates, or correcting inconsistencies.

The sales dataset is clean and well-prepared for analysis. It contains no duplicate records, and all relevant fields have been properly validated. Data types are consistent across columns, and there are no missing or null values. This ensures reliable and accurate insights can be derived from the data without the need for additional preprocessing.

Convert the data into a unified format, such as a common dataframe or data structure, to enable seamless analysis.

# Align columns if needed
common_columns = list(set(csv_data.columns) & set(excel_data.columns) & set(json_data.columns))

csv_data = csv_data[common_columns]
excel_data = excel_data[common_columns]
json_data = json_data[common_columns]

```
# Concatenate all into a single dataframe
all_sales_data = pd.concat([csv_data, excel_data, json_data], ignore_index=True)
# Final check
print(all_sales_data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 339108 entries, 0 to 339107
Data columns (total 18 columns):
 #   Column            Non-Null Count   Dtype
---  ------            --------------   -----
 0   Customer_Age      339108 non-null  int64
 1   Country           339108 non-null  object
 2   Month             339108 non-null  object
 3   Sub_Category      339108 non-null  object
 4   Profit            339108 non-null  int64
 5   State             339108 non-null  object
 6   Product           339108 non-null  object
 7   Cost              339108 non-null  int64
 8   Date              339108 non-null  object
 9   Day               339108 non-null  int64
 10  Product_Category  339108 non-null  object
 11  Unit_Cost         339108 non-null  int64
 12  Year              339108 non-null  int64
 13  Customer_Gender   339108 non-null  object
 14  Order_Quantity    339108 non-null  int64
 15  Age_Group         339108 non-null  object
 16  Unit_Price        339108 non-null  int64
 17  Revenue           339108 non-null  int64
dtypes: int64(9), object(9)
memory usage: 46.6+ MB
None
```

Data Transformation: merging datasets, splitting columns, deriving new variables

will only derive new variable

```
all_sales_data['Total_Sales'] = all_sales_data['Order_Quantity'] * all_sales_data['Unit_Price']
all_sales_data['Profit_Margin'] = all_sales_data['Profit'] / all_sales_data['Revenue']
```

```
all_sales_data.Profit_Margin
```

```
0         0.621053
1         0.621053
2         0.568930
3         0.568966
4         0.569378
             ...
339103    0.608696
339104    0.553677
339105    0.553677
339106    0.542857
339107    0.542668
Name: Profit_Margin, Length: 339108, dtype: float64
```

```
all_sales_data['Total_Sales']
```

```
0          960
1          960
2         2760
3         2400
4          480
           ...
339103     192
339104    1408
339105    1408
339106    1536
339107    1472
Name: Total_Sales, Length: 339108, dtype: int64
```

Analyze sales data: descriptive stats and aggregation

# a) Descriptive statistics
all_sales_data.describe()

	Customer_Age	Profit	Cost	Day	Unit_Cost	Year	Order_Quantity	Unit_Price
count	339108.000000	339108.000000	339108.000000	339108.000000	339108.000000	339108.000000	339108.000000	339108.000000
mean	35.919212	285.051665	469.318695	15.665753	267.296366	2014.401739	11.901660	452.938427
std	11.021903	453.886105	884.863508	8.781541	549.833862	1.272507	9.561829	922.068500
min	17.000000	-30.000000	1.000000	1.000000	1.000000	2011.000000	1.000000	2.000000
25%	28.000000	29.000000	28.000000	8.000000	2.000000	2013.000000	2.000000	5.000000
50%	35.000000	101.000000	108.000000	16.000000	9.000000	2014.000000	10.000000	24.000000
75%	43.000000	358.000000	432.000000	23.000000	42.000000	2016.000000	20.000000	70.000000
max	87.000000	15096.000000	42978.000000	31.000000	2171.000000	2016.000000	32.000000	3578.000000

b) Aggregate total sales by Product Category

sales_by_category = all_sales_data.groupby('Product_Category')
['Total_Sales'].sum().sort_values(ascending=False)
print(sales_by_category)

```
Product_Category
Bikes          207624588
Accessories     50137938
Clothing        27766428
Name: Total_Sales, dtype: int64
```

#c) Average order value

average_order_value = all_sales_data.groupby('Product')['Total_Sales'].mean()
print(average_order_value)

```
Product
AWC Logo Cap                    139.018816
All-Purpose Bike Stand          844.247788
Bike Wash - Dissolver           122.301552
Classic Vest, L                 936.050314
Classic Vest, M                 975.200000
                                   ...
Touring-3000 Yellow, 62         855.361111
Water Bottle - 30 oz.            76.007967
Women's Mountain Shorts, L     1033.720930
Women's Mountain Shorts, M     1090.921450
Women's Mountain Shorts, S     1046.951613
Name: Total_Sales, Length: 130, dtype: float64
```

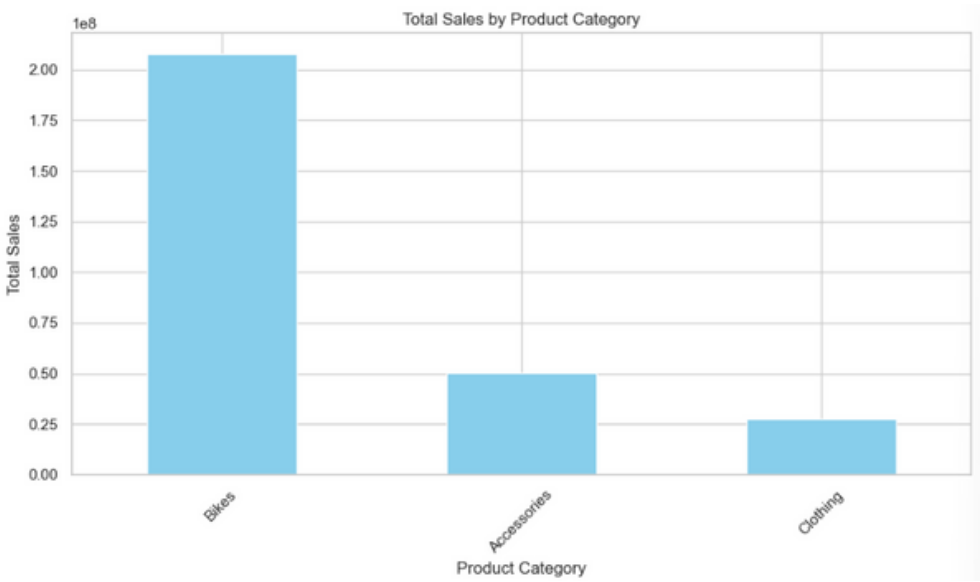
d) Distribution of product categories (counts)

category_counts = all_sales_data['Product_Category'].value_counts()
print(category_counts)

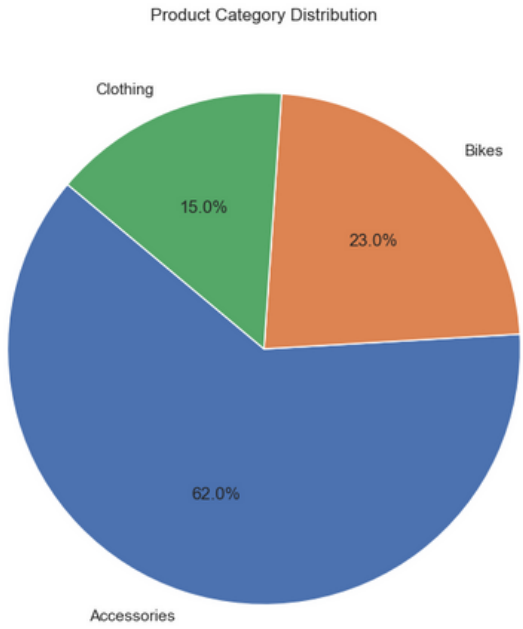
```
Product_Category
Accessories    210360
Bikes           77946
Clothing        50802
Name: count, dtype: int64
```

Create visualizations, such as bar plots, pie charts, or box plots, to represent the sales data and gain insights into sales trends, customer behavior, or product performance.

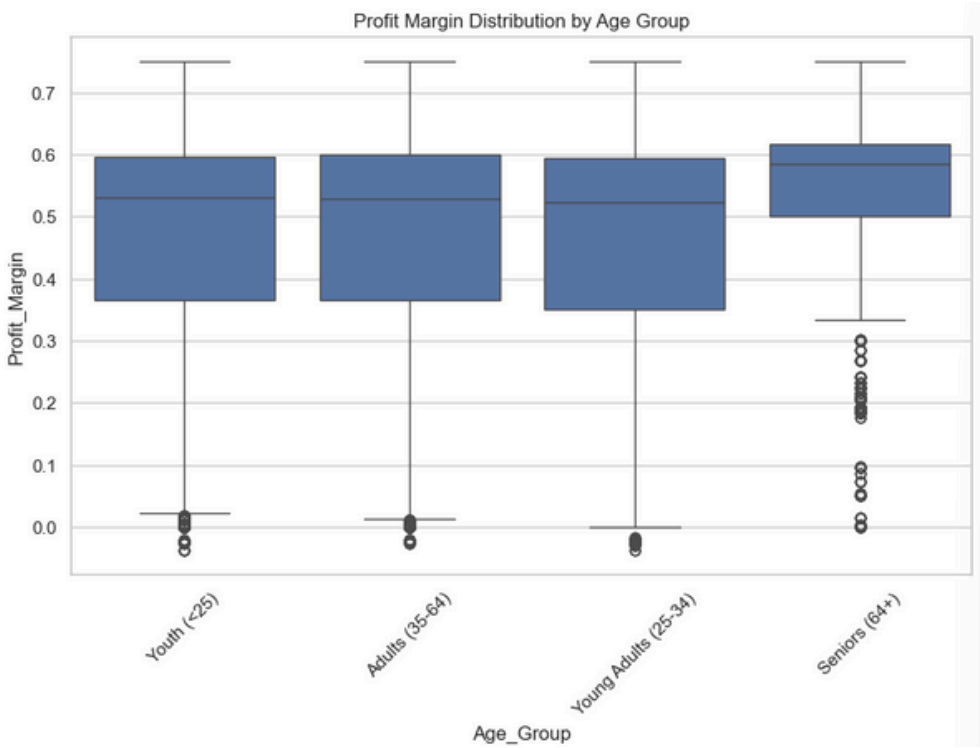
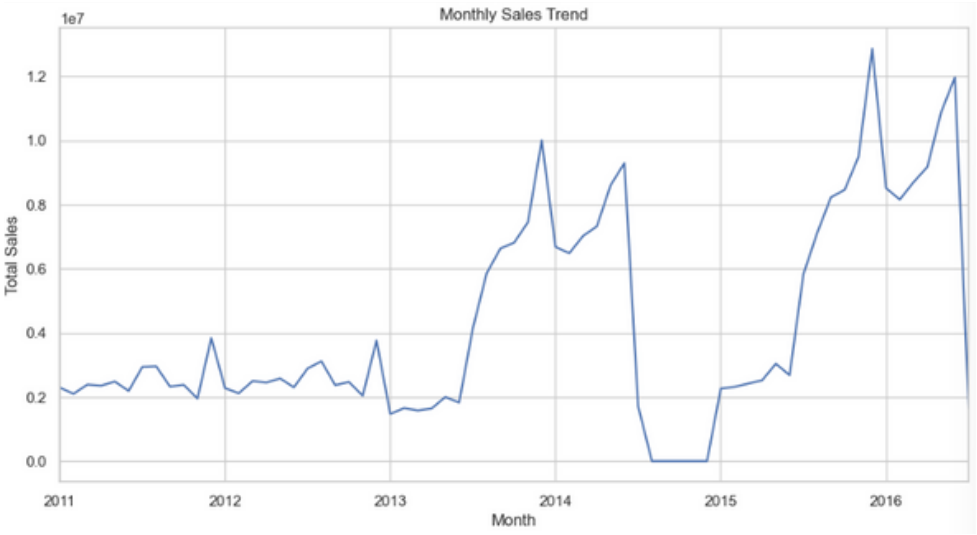
```
import matplotlib.pyplot as plt
import seaborn as sns

# Set style for plots
sns.set(style='whitegrid')

# a) Bar plot: Total Sales by Product Category
plt.figure(figsize=(10,6))
sales_by_category.plot(kind='bar', color='skyblue')
plt.title('Total Sales by Product Category')
plt.ylabel('Total Sales')
plt.xlabel('Product Category')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```




```
# b) Pie chart: Product Category distribution
plt.figure(figsize=(8,8))
category_counts.plot(kind='pie', autopct='%1.1f%%', startangle=140)
plt.title('Product Category Distribution')
plt.ylabel('')
plt.show()
```




```
# c) Box plot: Profit Margin by Age Group
plt.figure(figsize=(10,6))
sns.boxplot(data=all_sales_data, x='Age_Group', y='Profit_Margin')
plt.title('Profit Margin Distribution by Age Group')
plt.xticks(rotation=45)
plt.show()
```



```
# d) Line plot: Monthly sales trend over time

# Convert 'Date' column to datetime
all_sales_data['Date'] = pd.to_datetime(all_sales_data['Date'], errors='coerce')

# Drop rows where 'Date' could not be parsed
all_sales_data = all_sales_data.dropna(subset=['Date'])

# Group by month end and sum total sales
monthly_sales = all_sales_data.groupby(pd.Grouper(key='Date', freq='ME'))['Total_Sales'].sum()

# Plot
plt.figure(figsize=(12,6))
monthly_sales.plot(kind='line')
plt.title('Monthly Sales Trend')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.show()
```